

TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN A DISTANCIA - UTN

Unidad: De Java a Python: POO por Equivalencia

APUNTE ACTIVIDAD 1

El cambio de mentalidad y la tabla maestra

Índice

0 El cambio de mentalidad

Tres inversiones que explican casi todo lo demás

- Primera inversión · de la imposición al acuerdo
- Segunda inversión · de la herencia al duck typing
- Tercera inversión · de la declaración al runtime
- Por qué esto importa antes que la sintaxis

1 Tabla maestra de equivalencias

El mapa completo · concepto por concepto

0 El cambio de mentalidad

Tres inversiones que explican casi todo lo demás

Antes de cualquier tabla de equivalencias hay que entender tres inversiones conceptuales. Si las internalizás, la mayoría de las diferencias sintácticas se deducen solas y la tabla del capítulo 1 se vuelve casi innecesaria. Si no lo hacés, vas a escribir Java con sintaxis de Python durante años, y lo peor es que nadie te lo va a decir, porque el código funciona.

En Java	En Python
El compilador impone. El tipo es una barrera estática. <code>private</code> es una garantía.	El programador acuerda. El tipo es documentación. <code>_privado</code> es un pacto entre adultos.
Herencia para reusar tipo. Necesitás una superclase común para tratar objetos igual.	Duck typing. Si tiene el método, sirve. La superclase común es opcional.
La clase declara todo por adelantado.	El objeto se arma en runtime.

Primera inversión · de la imposición al acuerdo

En Java, cuando escribís `private int saldo`, estás haciendo una afirmación que el compilador va a hacer cumplir por vos. No es una sugerencia ni una nota al lector: es una barrera. Si alguien en otro paquete intenta tocar ese campo, el código no compila. Punto. El diseñador Java delega el cumplimiento de sus decisiones en una herramienta que no negocia.

En Python, cuando escribís `self._saldo`, estás haciendo exactamente la misma afirmación de diseño -«esto es interno, no lo toques»- pero no hay nadie que la haga cumplir. El guión bajo es un cartel. Un cartel que la comunidad entera respeta, que los linters marcan, que los code reviewers señalan, y que el intérprete ignora por completo. Si alguien escribe `cuenta._saldo = 999`, funciona.

El reflejo del programador Java al descubrir esto es concluir que Python «no tiene encapsulamiento». Es una conclusión comprensible y equivocada. El encapsulamiento es una *decisión de diseño*, no una característica del lenguaje. Java te da una herramienta para hacerla cumplir automáticamente; Python te pide que la hagas cumplir con convención, herramientas externas y cultura de equipo. La decisión -qué es interno y qué es público- la tomás vos en los dos casos, y es exactamente la misma decisión.

Lo que cambia es dónde vive el rigor. En Java vive en el lenguaje. En Python vive en el equipo. Ninguno de los dos modelos es intrínsecamente más disciplinado: hay bases de código Java con public en todos lados y bases de código Python con encapsulamiento impecable, y viceversa. Pero el modelo de Python es más honesto respecto de una verdad incómoda: el encapsulamiento de

Java tampoco es absoluto. Existe la reflexión, existe `setAccessible(true)`, y cualquier framework de serialización la usa todos los días. La diferencia es de grado, no de naturaleza.

Segunda inversión · de la herencia al duck typing

Esta es la que tiene consecuencias más profundas en el diseño, y la que se explica entera en el capítulo 6. Acá basta con plantear el punto.

En Java, si querés recorrer una lista de objetos y llamarles el mismo método, **necesitás** que tengan un tipo común. No es una recomendación de diseño: es un requisito del compilador. Sin una superclase o una interfaz compartida, no hay `List<Algo>` que los contenga, y sin esa lista no hay polimorfismo. Por eso, en Java, muchas jerarquías de herencia existen no porque el dominio diga que esas cosas son parientes, sino porque el compilador exige un tipo común para poder tratarlas igual.

En Python eso no hace falta. Si el objeto tiene el método, se le puede llamar. La lista puede contener cualquier cosa. El polimorfismo es gratis y no pide permiso.

Acá aparece la trampa, y es sutil: al descubrir esto, el programador Java tiende a concluir que en Python la herencia sobra. No sobra. Lo que sobra es la herencia *que existía solo para satisfacer al compilador*. La herencia que existe porque el dominio afirma que un triángulo es un polígono sigue estando plenamente justificada, y borrarla es empobrecer el modelo. La pregunta que hay que aprender a hacerse es: ¿esta superclase está acá porque el negocio dice que estas cosas son un mismo concepto, o está acá porque sin ella no compilaba? En Java las dos razones se confunden porque producen el mismo código. En Python se separan, y esa separación es la que revela cuál era cuál.

Tercera inversión · de la declaración al runtime

En Java, una clase es un contrato cerrado en tiempo de compilación. Los campos que declaraste son los campos que hay. Los métodos que escribiste son los métodos que existen. Un objeto de esa clase no puede, en ejecución, adquirir un atributo que no estaba previsto.

En Python, la clase es un punto de partida y el objeto se termina de armar en ejecución. Podés agregarle atributos a una instancia después de creada. Podés reemplazar un método. Podés preguntarle a un objeto qué métodos tiene y decidir en runtime. Esta flexibilidad es la base de gran parte del ecosistema -los ORMs, los frameworks de test, la serialización- y es también la razón por la que un typo en un nombre de atributo no produce un error de compilación sino un objeto silenciosamente distinto del que esperabas.

El corolario práctico es que en Python muchas cosas que Java resuelve en tiempo de compilación se resuelven con herramientas: mypy para los tipos, los linters para las convenciones, los tests para el resto. La red existe, pero no viene puesta: hay que instalarla.

Por qué esto importa antes que la sintaxis

La inversión de fondo, la que unifica a las tres, es esta: **Java pone el conocimiento del diseño en el lenguaje** -el modificador, la firma, la anotación, la declaración- mientras que **Python lo pone en el equipo**: la convención, el linter, el type checker, el code review, el manual de la cátedra.

Ninguno de los dos es más riguroso que el otro. Cambia dónde vive el rigor. Y como cambia de lugar, un programador que trae el hábito de un lenguaje al otro sin darse cuenta de la mudanza termina buscando el rigor donde ya no está -y no encontrándolo- o dejando de ejercerlo porque el compilador ya no se lo reclama.

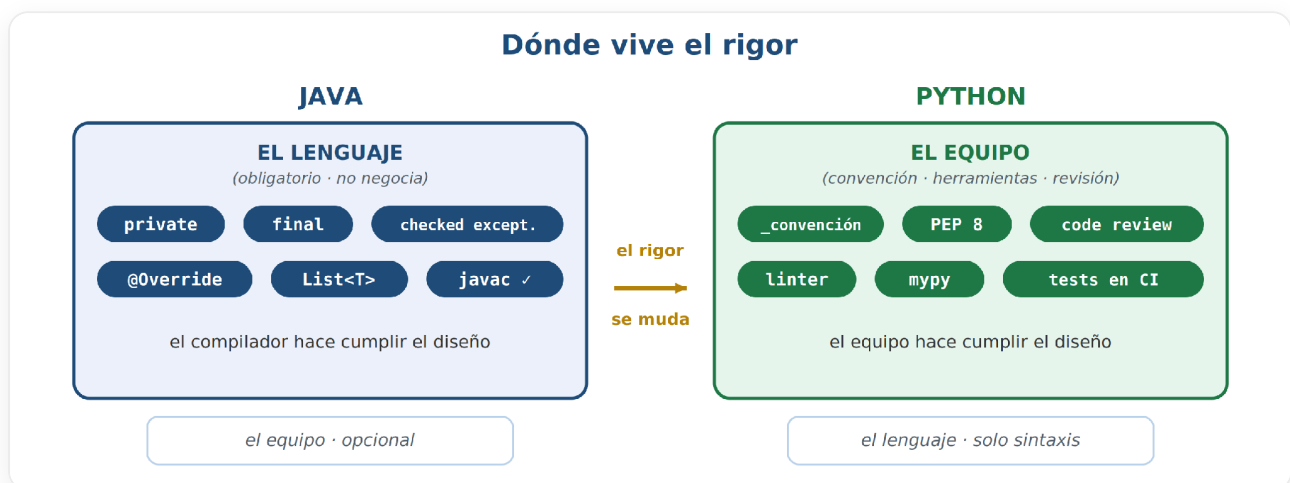


Figura 2 · La mudanza del rigor: en Java lo hace cumplir el lenguaje; en Python, el equipo y sus herramientas. La decisión de diseño es la misma.

Con esto en la cabeza, la tabla del próximo capítulo se lee distinto. No es una lista de traducciones: es un mapa de dónde el diseño se conserva idéntico y dónde hay que volver a pensarlo.

1 Tabla maestra de equivalencias

El mapa completo · concepto por concepto

Este capítulo es referencia. No está pensado para leerse de corrido sino para consultarse mientras programás, y por eso está partido en cuatro tablas temáticas en lugar de una sola de cuarenta filas.



Cada **No** de estas tablas tiene un capítulo detrás que lo explica.

El mapa de la tabla maestra

la columna «¿Exacta?» divide el pasaje en tres zonas

SÍ — traducción directa

cambia la sintaxis y nada más

clase

herencia

ABC

excepciones

__str__

static

CUIDADO — parecido, con trampa

la forma coincide; el comportamiento no del todo

super()

__eq__/__hash__

streams

campo de clase

self

NO — cambio de modelo mental

acá se cometen los errores: hay un capítulo detrás

private

sobrecarga

interfaz

@Override

genéricos

checked exc.

Figura 3 · Las tres zonas del pasaje. La verde se cruza sin pensar; la roja es donde un hábito Java produce un error Python.

Estructura de clase

Concepto POO	Java	Python	¿Equivalencia Exacta?
Clase	<code>class Figura { }</code>	<code>class Figura:</code>	Sí
Constructor	<code>Figura(String n) { }</code>	<code>def __init__(self, n: str) -> None:</code>	Sí, pero <code>__init__</code> inicializa, no crea (eso es <code>__new__</code>)
Referencia al objeto	<code>this</code> -Implícito	<code>self</code> - explícito, siempre como 1er parámetro	Conceptual sí, sintáctico no
Herencia	<code>class Triangulo extends Poligono</code>	<code>class Triangulo(Poligono):</code>	Sí
<code>super()</code>	<code>super(n);</code>	<code>super().__init__(n)</code>	Sí - pero NO es automático
Clase abstracta	<code>abstract class Poligono</code>	<code>class Poligono(ABC):</code>	Sí
Método abstracto	<code>abstract int ladosEsperados();</code>	<code>@abstractmethod</code>	Sí
Interfaz	<code>interface Comparable { }</code>	Protocol / ABC pura	No es exacto - ver cap. 6
<code>@Override</code>	Anotación verificada	(nada)	No existe. Typo silencioso

Dos filas de esta tabla merecen comentario porque son fuente habitual de bugs.

El `super()` no es automático. En Java, si tu constructor no llama explícitamente a `super(...)`, el compilador inserta una llamada al constructor sin argumentos del padre. Es invisible pero está. En Python no hay nada de eso: si no escribís `super().__init__()`, el constructor del padre simplemente no corre. El objeto queda a medio construir, sin los atributos que el padre iba a inicializar, y el error aparece mucho después -en otro método, en otro archivo- cuando algo intenta leer un atributo que nunca existió. Es de los bugs más desorientadores del pasaje.

El `@Override` no existe. En Java, la anotación le pide al compilador que verifique que efectivamente estás sobrescribiendo algo. Si te equivocás en el nombre o en la firma, no compila. En Python no hay equivalente: si querés redefinir `calcular_area` y escribís `calcular_aera`, acabás de crear un método nuevo. El padre sigue respondiendo con su implementación, tu método nunca se llama, y no hay ningún error en ninguna parte. Python 3.12 agregó `@typing.override`, que `mypy` verifica - pero es opcional y hay que acordarse de ponerlo.

PYTHON - EL TYPO SILENCIOSO Y SU RED

```
class Poligono:

    def calcular_area(self) -> float: ...

class Triangulo(Poligono):

    def calcular_aera(self) -> float: # typo: creó un método NUEVO
    return self.base * self.altura / 2
    \# Triangulo().calcular_area() llama al del padre. Sin error, nunca.
    \# Desde Python 3.12, la red equivalente al @Override:
        from typing import override

class Triangulo(Poligono):

    @override
    def calcular_aera(self) -> float: # ahora mypy Sí lo marca
    ...
```

Visibilidad y estado

Concepto POO	Java	Python	¿Equivalencia Exacta?
public	public int x;	self.x	Sí (default)
protected	protected int x;	self._x - convención	No. Nada lo impide
private	private int x;	self.__x - name mangling	No. Solo ofusca
final (campo)	final List<Lado> lados	Final[list[Lado]] o property sin setter	No. Es mas anotación, no implica restricción
static	static int contar()	@staticmethod	Sí
Campo de clase	static int total;	total: int = 0 (nivel clase)	Sí - ojo mutables (cap. 5)
Getter / Setter	getNroLados() / setX()	@property / @x.setter	Mejor en Python - cap. 2

Esta tabla es la más roja del manual, y no es casualidad: la visibilidad es donde Java y Python discrepan más. Tres de las siete filas dicen que no hay equivalencia real. El capítulo 3 se ocupa entero de esto, pero conviene registrar desde ahora que **ninguna construcción de Python reproduce el private de Java** - ni el guión bajo simple, ni el doble, ni ninguna combinación. Lo

más parecido que existe es una property de solo lectura que devuelve una copia, y ni siquiera eso impide el acceso al atributo interno.

Comportamiento y tipos

Concepto OO	Java	Python	¿Equivalencia Exacta?
Sobrecarga	area(int), area(int,int)	Default args, *args, @singledispatch	No existe
Genéricos	List<Lado>	list[Lado] - hint, no verificado	No. Documentación + mypy
toString()	@Override public String toString()	__repr__ / __str__	Sí
equals()/hashCode()	Par obligatorio	__eq__ / __hash__	Sí - misma trampa del par
Comparable	compareTo()	__lt__ + @total_ordering	Sí
Excepción	throw new IllegalArgumentException(m)	raise ValueError(m)	Sí
Checked exceptions	throws IOException	(no existe)	No. Ninguna es checked
Herencia múltiple	Solo de interfaces	De clases, permitida (MRO/C3)	Python puede más

La sobrecarga no existe, y esto sorprende. Si definís dos métodos con el mismo nombre y distinta firma, el segundo pisa al primero. Sin error, sin advertencia. Python resuelve las llamadas por nombre, no por firma, así que el concepto de «misma operación, distintos parámetros» se expresa de otra manera: con argumentos por defecto cuando las variantes son pocas y compatibles, con *args cuando la cantidad es variable, con @classmethod cuando lo que querés son constructores alternativos -el caso más frecuente- y con @singledispatch cuando el comportamiento realmente depende del tipo del argumento. El capítulo 4 muestra el patrón.

La trampa del par equals/hashCode se conserva idéntica. En Java, si sobrescribís equals sin sobrescribir hashCode, tu objeto se comporta mal en cualquier HashMap o HashSet. En Python la trampa es la misma pero con un agravante: si definís __eq__ y no definís __hash__, Python *anula* el hash - tu clase pasa a ser no hasheable y revienta al meterla en un set. Es la misma regla de diseño de siempre, con un castigo más ruidoso.

Colecciones y organización

Concepto OO	Java	Python	¿Exacta?
ArrayList<T>	new ArrayList<>()	[]	Sí
Map<K,V>	new HashMap<>()	{}	Sí
Streams	.stream().filter().map()	Comprehensions	Conceptual sí - cap. 4
Paquete	package ar.utn.figuras;	Módulo = archivo .py	Aproximado

La última fila dice «aproximado» por una razón que conviene tener presente al organizar un proyecto: en Java el paquete es una jerarquía de directorios que se corresponde con el nombre, y una clase pública vive en su propio archivo. En Python el módulo es el archivo, y un archivo puede contener tantas clases como tenga sentido agrupar. Un módulo `figuras.py` con `Figura`, `Poligono`, `Triangulo` y `Lado` no es desprolijidad: es la organización idiomática. Traer la regla de «una clase, un archivo» desde Java produce proyectos Python fragmentados en decenas de archivos de quince líneas, y es uno de los java-ismos más visibles a simple vista.